GUIA TÉCNICO | 18

Redes recorrentes, LSTM e GRU

Capítulo 18/20 · Fundamentos, experiência reproduzível e interpretação

PROPÓSITO E OBJETIVOS

- Explicar estados ocultos, dependências temporais e controle por portas.
- Relacionar a técnica à sua função de perda, representação e protocolo de avaliação.
- Explicar limitações e formular um teste reprodutível.

MAPA DE ESTUDO

1. Fundamentação do método
2. Expressões e hipóteses
3. Experimento essencial executável
4. Laboratório aplicado e limitações
5. Exercício e verificações

01 FUNDAMENTAÇÃO CONCEITUAL

RNNs processam sequências com estado oculto compartilhado. O gradiente pode desaparecer/explodir em sequências longas; LSTM e GRU usam portas para reter e atualizar informação. Shapes e máscaras distinguem sequências de comprimentos variáveis.

Fonte histórica de apoio: notas do curso original, consultadas apenas como referência de contexto; as explicações e exemplos desta edição são independentes.

Questões críticas: quais pressupostos são necessários? O que a função aprende? Qual métrica indica utilidade real? Que tipo de vazamento ou má interpretação invalidaria a conclusão?

02 MATEMÁTICA E DIMENSÕES

Expressão esquemática: h_t=tanh(W_x x_t+W_h h_(t−1)+b)

A expressão acima resume a relação central. Para a derivação, identifique dimensões, hipóteses e operações; não interprete notação compacta como uma implementação completa.

LEITURA CRÍTICA DO CAPÍTULO

Estados recorrentes, gradientes no tempo e portas de memória ajudam a explicar RNN/LSTM/GRU e suas limitações.

Procedimento: execute a próxima célula com o núcleo declarado em requirements.txt, confira os assert e interprete os resultados. O experimento é independente dos exemplos anteriores.

03 EXPERIMENTO ESSENCIAL

Código extraído do notebook revisado. Execute a célula completa no JupyterLab; as quebras visuais abaixo não substituem o arquivo executável.

```
import numpy as np
rng=np.random.default_rng(0)
x=np.array([.2,.3,.4]);h=0.;wx=.8;wh=.35
estados=[]
for t in x:
 h=np.tanh(wx*t+wh*h);estados.append(h)
print('Estados da RNN escalar:',np.round(estados,4).tolist())
assert len(estados)==3
```

INTERPRETAÇÃO

Compare o resultado obtido com a formulação matemática. Confirme shapes, escala e hipótese de modelagem. Um experimento sintético valida uma operação, não substitui avaliação em dados

reais. Ao treinar modelos, separe dados e ajuste transformações apenas no treino; preserve um conjunto final de teste.

04 LABORATÓRIO APLICADO

Laboratório aplicado revisado — Sensibilidade a estados iniciais em RNN escalar

Estados recorrentes, gradientes no tempo e portas de memória ajudam a explicar RNN/LSTM/GRU e suas limitações.

Procedimento: execute a próxima célula com o núcleo declarado em requirements.txt, confira os assert e interprete os resultados. O experimento é independente dos exemplos anteriores.

```
import numpy as np
seq=np.ones(25)*.1
def rnn(h0,wh):
 h=h0
 for x in seq:h=np.tanh(.2*x+wh*h)
 return h
h=1e-5
for wh in [.2,.6,.95]:
 derivative=(rnn(h,wh)-rnn(-h,wh))/(2*h)
 print('peso recorrente',wh,'sensibilidade ao estado inicial',round(derivative,8))
assert abs(rnn(h,.2)-rnn(-h,.2))<.1
```

É um experimento ilustrativo, não diagnóstico universal de gradientes: saturação de tanh e pesos recorrentes podem alterar o comportamento.

Checklist: conferir entradas e saídas, sementes, inexistência de uso dos dados de teste durante ajuste e distinção entre uma demonstração sintética e desempenho em aplicações reais.

05 EXERCÍCIO E SOLUÇÃO COMENTADA

Por que LSTM pode superar RNN simples em sequências longas?

Resposta: Suas portas e caminho de memória ajudam a preservar informação e reduzir o desaparecimento de gradientes.

LIMITAÇÕES E BOAS PRÁTICAS

- Confundir ajuste no treino com generalização para dados não vistos.
- Reportar somente uma métrica ou uma execução aleatória.
- Ignorar shape, tipo de variável e convenção do backend.
- Executar exemplos com pesos baixados sem registrar versão, licença e procedência.
- Confundir demonstração matemática com validação de um modelo de produção.

VERIFICAÇÃO LOCAL

Execute Reiniciar kernel e executar todas as células. Registre versões do Python e das dependências, confira shapes e métricas e diferencie os laboratórios opcionais dos exemplos NumPy/scikit-learn.

Validação posterior (13/11/2026): notebooks 14 a 20 executados com TensorFlow CPU; opcionais 01 a 13 não integralmente revalidados.

REFERÊNCIAS TÉCNICAS

NumPy: numpy.org/doc · scikit-learn: scikit-learn.org/stable/user_guide.html · Keras: keras.io/guides · TensorFlow: tensorflow.org/guide